In [3]:
```
#print 1-n using recursion
class solution:
    def printNumbers(self,i:int,n:int)->int:
        #Base Case
        if i>n:
            return
        #Recursive Case
        print(i,end=' ')
        self.printNumbers(i+1,n)
obj=solution()
obj.printNumbers(1,5)
```

```
1 2 3 4 5
```

In [5]:
```
#Factorial of a number
def Factorial(n):
    if n==0:
        return 1
    return n*Factorial(n-1)
Factorial(5)
```

Out[5]: 120

In [ ]:
```
#Recursive Stack: It follows LIFO[Last In First Out] principle.
def fun(n):
    if n==0:
        return
    print(n,end=" ")
    fun(n-1)
fun(3)

print()

def fun(n):
    if n==0:
        return
    fun(n-1)
    print(n,end=" ")
fun(3)
```

```
3 2 1
1 2 3
```

In [ ]:
```
#Fibanocci --> TIme complexity:O(2**n), space complexity:
def fib(n):
    if n==0:
        return 0
    if n==1:
        return 1
    return fib(n-1)+fib(n-2)
fib(10)
```

Out[ ]: 55

In [ ]:
```
#Power of 2
#Idea--> 1,2,4,8,16,32,64,128... are power of 2, divide them with 2 till we get
class solution:
    def IsPowerOfTwo(self,n:int)->bool:
```

```
        if n<0:
            return False
        if n==1:
            return True
        if n%2!=0:
            return False

        return self.IsPowerOfTwo(n//2)
obj=solution()
obj.IsPowerOfTwo(16)

print()
#Power of 3
class solution:
    def IsPowerOfThree(self,n:int)->bool:
        if n<0:
            return False
        if n==1:
            return True
        if n%3!=0:
            return False

        return self.IsPowerOfThree(n//3)
obj=solution()
obj.IsPowerOfThree(81)
```

Out[ ]: True

In [ ]:
```
#Greatest Common Divisor-->Euclidean Method
def GCD(a,b):
    if b==0:
        return a
    return GCD(b,a%b)
print(GCD(15,50))
```

```
5
```

In [18]:
```
#NOTE:a*b=GCD(a,b)*LCM(a,b)
#LCM(a,b)=a*b/GCD(a,b)

class solution:
    def GCD(slef,a,b):
        if b==0:
            return a
        return GCD(b,a%b)
    def LCM(self,a,b):
        return (a*b)/self.GCD(a,b)
obj=solution()
r1=obj.GCD(15,50)
r2=obj.LCM(15,50)
print(f'GCD(15,50) is:{r1}')
print(f'LCM is(15,50):{r2}')
```

```
GCD(15,50) is:5
LCM is(15,50):150.0
```

In [ ]:
```
#power--> X^n
class solution:
```

```
    def Power(self,x,n):
        if n==0:
            return 1
        half= self.Power(x,n//2)

        if n%2!=0:
            return half*half*x
        else:
            return half**2

obj=solution()
obj.Power(2,5)
```

Out[ ]: 32

The Kernel crashed while executing code in the current cell or a previous cell.

Please review the code in the cell(s) to identify a possible cause of the failur
e.

Click here for more info.

View Jupyter log for further details.